GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY
Faculty of Computing | CS31092 — Digital Image Processing

Face Verification Attendance System

Technical Documentation & Code Explanation

Group 04
D/BCE/24/0007 DMMH Dissanayake
D/BCS/24/0010 HDT Gajanayaka
D/DBA/23/0012 OM Kavishan

2025

1. Project Overview

The Face Verification Attendance System is a real-time, automated attendance solution developed for General Sir John Kotelawala Defence University (KDU). The system eliminates manual attendance by using a live webcam feed to detect and identify student faces, recording verified attendance directly into a MongoDB database.

The core of the system is an 11-step image preprocessing pipeline that transforms every raw camera frame into a clean, normalized input for a deep learning model. A pretrained ResNet-34 Convolutional Neural Network then encodes each face into a 128-dimensional mathematical vector, which is compared against stored encodings using Euclidean distance to verify identity.

1.1 The Problem Being Solved

Manual Attendance Problem	Our System's Solution
Slow and disrupts class time	Fully automated — runs silently in background
Paper-based, easy to manipulate	Face biometrics — cannot be faked with a name
Error-prone data entry	Direct MongoDB insertion — no human errors
No real-time records	Instant timestamp logging with subject tagging
Duplicate entries possible	60-second deduplication window enforced

1.2 Technologies & Libraries

Technology	Role	Why Chosen
Python 3.x	Core backend language	Vast ML/CV library ecosystem
Flask	Web framework & routing	Lightweight, easy Python integration
OpenCV (cv2)	Camera capture, resize, draw	Industry standard for real-time CV
face_recognition	Face detection & encoding	Wraps dlib's ResNet-34 — high accuracy
dlib	Underlying ML engine	Efficient C++ landmark detection
NumPy	Array math & distance calc	Vectorised operations — very fast
MongoDB	NoSQL attendance database	Flexible schema for timestamped logs
PyMongo	Python–MongoDB connector	Official MongoDB Python driver

2. System Architecture & Workflow

The system operates in two distinct modes: Registration Mode (admin captures student face) and Verification Mode (live attendance). Both share the same preprocessing pipeline.

2.1 Registration Mode

Stage	What Happens
1. Admin Login	Admin authenticates with hardcoded credentials
2. Camera Preview	Live MJPEG stream shown on registration page
3. Photo Capture	Admin clicks capture → single frame saved as JPEG
4. File Naming	Saved as Name--StudentID--Subject.jpg in images/ folder
5. Encoding Load	load_known_faces() re-runs to include the new student

2.2 Verification Mode — Live Attendance

Stage	Technique Used	Details
Frame Capture	cv2.VideoCapture	CAP_DSHOW on Windows
BGR → RGB	cv2.cvtColor	Channel order correction
Resize to 25%	cv2.resize fx=0.25	Bilinear interpolation
CLAHE	cv2.createCLAHE	LAB space, 8×8 tiles, clip=2.0
Gaussian Blur	cv2.GaussianBlur	3×3 kernel, σ=0.5
HOG Detection	face_recognition.face_locations	model='hog', SVM classifier
ResNet-34 Encode	face_recognition.face_encodings	128-D float64 vector
Euclidean Match	face_recognition.face_distance	Threshold ≤ 0.50
MongoDB Log	attendance_collection.insert_one	name + subject + timestamp

3. Image Preprocessing Pipeline — Detailed Explanation

Image preprocessing is the most technically critical phase. Without it, the face recognition model fails in real-world university conditions. Each step solves a specific problem in the raw camera input. All 11 steps run on every camera frame in real time.

Why Preprocessing Matters

The raw camera feed has inconsistent lighting (windows, fluorescent lights), motion blur, sensor noise, head pose variation, and channel ordering differences. Each preprocessing step resolves one or more of these issues, transforming an unreliable raw frame into a clean, normalized input that ResNet-34 can accurately encode.

STEP 1 — Raw Frame Capture

Acquiring the live image as a NumPy array

OpenCV's VideoCapture continuously reads frames from the hardware webcam and returns them as NumPy arrays with shape (Height × Width × 3). The three channels are Blue, Green, Red (BGR), each stored as 8-bit unsigned integers (values 0–255).

```
ret, frame = camera.read()
# frame.shape → (480, 640, 3)   height × width × channels
# frame.dtype → uint8           values 0 to 255
```

Windows Fix — CAP_DSHOW Flag

On Windows, the default MSMF (Microsoft Media Foundation) backend causes black frames or fails entirely with many laptop webcams. The cv2.CAP_DSHOW flag switches OpenCV to the DirectShow API which has broader Windows driver support. This is the most important Windows-specific fix in the codebase: cv2.VideoCapture(0, cv2.CAP_DSHOW)

STEP 2 — BGR → RGB Color Space Conversion

Correcting channel order mismatch

OpenCV stores all images in BGR (Blue-Green-Red) order — a legacy from early video hardware. The face_recognition library and the ResNet-34 model were both trained on RGB (Red-Green-Blue) images from the Labeled Faces in the Wild (LFW) dataset. If BGR is passed instead of RGB, the red and blue channels are swapped for every pixel. A person with warm skin tone (high R in RGB) appears cool-toned (high B in BGR). The CNN produces a completely wrong 128-D embedding and face matching fails entirely.

```
frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
# Equivalent to: frame_rgb = frame_bgr[:, :, ::-1]
# BGR: [:,:,0]=Blue  [:,:,1]=Green  [:,:,2]=Red
# RGB: [:,:,0]=Red   [:,:,1]=Green  [:,:,2]=Blue
```

Critical Rule

BGR→RGB conversion MUST happen BEFORE calling face_recognition.face_locations() and face_recognition.face_encodings(). Both functions assume RGB input. Skipping this step causes silent encoding failures — the system runs without errors but identifies nobody correctly.

STEP 3 Spatial Resizing — Scale to 25%

Reducing resolution for real-time performance

HOG face detection uses a sliding window approach: a detection window moves across the image at multiple scales, computing a HOG descriptor at every position. On a 640×480 frame this involves thousands of window positions. On a 1080p frame it involves tens of thousands. Running this at 30 FPS is computationally impossible without resizing. Scaling to 25% reduces pixel count by 94%, making real-time detection feasible.

```
small_frame = cv2.resize(frame_rgb, (0, 0), fx=0.25, fy=0.25)
# (0,0) → output size computed from scale factors
# fx=0.25 → width × 0.25    fy=0.25 → height × 0.25
# Default interpolation: INTER_LINEAR (bilinear)
```

Interpolation Method	How It Works	Used For
INTER_NEAREST	Copies nearest pixel — fastest, pixelated	Thumbnails
INTER_LINEAR (ours)	Weighted average of 2×2 neighbourhood — smooth	Face detection — our choice
INTER_CUBIC	Weighted average of 4×4 neighbourhood — sharper	Offline/high-quality processing

Scale-Back Rule

After detecting faces on the small frame, all bounding box coordinates must be multiplied by 4 before drawing on the original full-resolution frame. Example: a face at top=30, right=80 on the small frame is at top=120, right=320 on the original.

STEP 4 Grayscale Conversion

Reducing 3-channel colour to single intensity channel

HOG detection operates on image intensity gradients — the magnitude and direction of brightness change between pixels. Colour information is irrelevant to this computation. Converting to grayscale reduces each pixel from a 3-element [R,G,B] vector to a single scalar intensity value, reducing processing by 3×. Importantly, grayscale is NOT a simple average — it uses a perceptual weighted formula based on human visual sensitivity.

```
gray = cv2.cvtColor(small_frame, cv2.COLOR_RGB2GRAY)
# Formula: Gray = 0.299×R + 0.587×G + 0.114×B
# Green highest weight (0.587): most sensitive to human eye
# Blue lowest weight (0.114): least sensitive to human eye
```

Note

The face_recognition library handles grayscale conversion internally. You do not need to do this manually. It is explained here to understand what happens inside face_locations() when you call it.

STEP 5 — CLAHE — Lighting Normalization

Adaptive histogram equalization for uneven classroom lighting

One of the biggest real-world problems is inconsistent lighting. A student near a window may be backlit (face appears very dark). A student under a bright lamp may have blown-out highlights. In both cases, the face's gradient structure is compressed into a narrow intensity range, making HOG descriptors weak and detection unreliable.

Why Not Basic Histogram Equalization?

Basic histogram equalization works globally — it redistributes all pixel intensities across the full 0–255 range. This over-brightens already-lit regions and amplifies noise in the background. CLAHE (Contrast Limited Adaptive Histogram Equalization) is the correct solution.

How CLAHE Works

- Divides the image into small 8×8 pixel tiles
- Applies histogram equalization independently to each tile
- clipLimit=2.0 prevents any bin from growing too large (prevents noise amplification)
- Neighbouring tiles are blended (interpolated) to eliminate artificial tile boundaries
- Applied only to the L (Luminance) channel in LAB colour space — colour information unchanged

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
lab   = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2LAB)
l, a, b = cv2.split(lab)
l_eq  = clahe.apply(l)          # Only brighten L channel
result = cv2.merge([l_eq, a, b]) # Colour channels A, B untouched
equalized_rgb = cv2.cvtColor(result, cv2.COLOR_LAB2RGB)
```

LAB Colour Space — Why It Matters

LAB separates colour (A=green↔red, B=blue↔yellow) from brightness (L=Luminance). Applying CLAHE only to L preserves natural skin tones and eye colours while correcting only the brightness distribution. Equalising each RGB channel independently would create severe colour distortion — faces would appear green or purple.

STEP 6 — Gaussian Blur — Noise Reduction

Removing sensor noise to prevent false gradient edges

Camera sensors introduce random noise — pixels that should be 120 may read as 117 or 124 randomly. These create small spurious intensity gradients that HOG interprets as facial features, leading to false detections or degraded descriptors. Gaussian blur averages each pixel with its neighbours using weights that follow a 2D Gaussian (bell curve) distribution, smoothing noise while preserving real edges.

```
blurred = cv2.GaussianBlur(frame, (3, 3), sigmaX=0.5)
# (3,3)    → kernel size: 3×3 pixel averaging window
# sigmaX=0.5 → how sharply weight falls off from centre
# A 3×3, σ=0.5 is a mild blur — removes noise, keeps face edges
```

Blur Filter	Strength	Best For
GaussianBlur(3,3,0.5) — our choice	Mild	General sensor (Gaussian) noise
medianBlur(frame, 3)	Strong edge preservation	Salt-and-pepper noise
bilateralFilter	Best edge preservation, slowest	Offline processing only

STEP 7 HOG Face Detection

Locating face bounding boxes using Histogram of Oriented Gradients + SVM

With the frame preprocessed, face detection can now run. The face_recognition library uses dlib's HOG-based detector — a combination of the Histogram of Oriented Gradients (HOG) feature descriptor and a linear Support Vector Machine (SVM) classifier trained on thousands of face and non-face images.

5-Stage HOG Detection Process

Stage	What Happens	Technical Detail
1. Gradient Computation	Compute Gx, Gy for every pixel	Sobel operators (3×3 kernels). magnitude=$\sqrt{(Gx^2+Gy^2)}$, direction=arctan(Gy/Gx)
2. Cell Histograms	Divide image into 8×8 cells	Build 9-bin orientation histogram per cell (bins: 0°,20°,40°…160°). Weight by magnitude
3. Block Normalisation	Group 4 cells into 16×16 block	L2-normalise the 36-element descriptor (4 cells × 9 bins). Makes it lighting-robust
4. SVM Classification	Slide 64×128 detection window	8-pixel strides. HOG descriptor fed to linear SVM → face(1) or non-face(−1)
5. Image Pyramid	Repeat at multiple scales	Downsample by 1.05× each level. Non-max suppression merges overlapping detections

```
face_locations = face_recognition.face_locations(small_frame, model='hog')
# Returns: [(top, right, bottom, left), ...]  — note order!
# Scale back to full resolution:
top, right, bottom, left = [coord*4 for coord in face_location]
```

HOG vs CNN Detection

model='hog' (our choice) — Fast, works in real time at 30 FPS. Suitable for well-lit environments with frontal faces. model='cnn' — Uses a Convolutional Neural Network detector. Much more accurate for angled faces, partial occlusions, and dim lighting. However, 5–10× slower. Recommended for the registration capture where accuracy matters more than speed.

STEP 8 ROI Crop — Region of Interest Extraction

Isolating the face region from the full frame

Once the face bounding box is known, we crop only the face from the full-resolution RGB frame. This Region of Interest (ROI) contains the face and nothing else — no background furniture, other students, or irrelevant content. Passing only the face region to the ResNet-34 encoder is more efficient and prevents the model from incorporating non-face features into the embedding.

```
top, right, bottom, left = face_location  # On small frame
t, r, b, l = top*4, right*4, bottom*4, left*4  # Scale up
# Padded crop (20px border — recommended):
pad = 20
face_roi = frame_rgb[max(0,t-pad):min(H,b+pad), max(0,l-pad):min(W,r+pad)]
```

Why Add Padding?
The HOG bounding box tightly encloses the face but may cut off the chin, forehead, or ears. ResNet-34 uses facial boundary features (jaw shape, hairline) that could be missed with a too-tight crop. A 20px border at full resolution provides enough context without including too much background.

STEP 9 Face Alignment — Pose Normalisation

Rotating face to horizontal using eye landmark positions

When a student tilts their head, the spatial positions of facial features shift relative to the image frame. The ResNet-34 model produces more consistent embeddings when the face is geometrically aligned — left eye always in approximately the same position. The face_recognition library uses dlib's 5-point facial landmark predictor (two eye corners, one nose tip, two mouth corners) to compute an affine transformation matrix that rotates and translates the face to a canonical horizontal position.

```
landmarks = face_recognition.face_landmarks(frame_rgb, face_locations)
# Returns: {'left_eye': [(x,y),...], 'right_eye': [(x,y),...], ...}
left_eye_center  = np.mean(landmarks['left_eye'],  axis=0)
right_eye_center = np.mean(landmarks['right_eye'], axis=0)
angle = np.degrees(np.arctan2(dy, dx))  # Rotation angle
M     = cv2.getRotationMatrix2D(face_center, angle, 1.0)
aligned = cv2.warpAffine(frame_rgb, M, (W, H))
```

Note
face_recognition handles alignment internally inside face_encodings(). The code above is shown so you understand the mathematics. You do not need to call it manually.

STEP 10 Pixel Normalisation

Scaling pixel values to the range expected by the CNN

Neural networks are sensitive to the scale of their input values. ResNet-34's weights and biases were tuned during training with normalised inputs. Feeding raw 0–255 values causes the first layer's activations to be extremely large, creating numerical instability and producing embeddings inconsistent with training-time behaviour.

Two-Stage Normalisation

- Stage 1 — Divide by 255: Scale pixel values from [0, 255] to [0.0, 1.0]
- Stage 2 — ImageNet Normalisation: Subtract the per-channel mean and divide by standard deviation of the ImageNet training dataset

```
normalized = frame.astype(np.float32) / 255.0
imagenet_mean = [0.485, 0.456, 0.406]  # R, G, B channel means
imagenet_std  = [0.229, 0.224, 0.225]  # R, G, B channel std devs
normalized = (normalized - imagenet_mean) / imagenet_std
```

Why ImageNet Values?

The ResNet-34 backbone was first pretrained on the ImageNet dataset (1.2 million images), then fine-tuned for face recognition. ImageNet's mean=[0.485, 0.456, 0.406] and std=[0.229, 0.224, 0.225] represent the average colour distribution across those 1.2M training images. Applying the same normalisation at inference time ensures pixel values are in the same statistical range the model was trained on.

Warning

face_recognition handles both normalisation stages internally. Do NOT manually normalise images before passing them to face_encodings() — double-normalisation produces incorrect embeddings and matching will fail silently.

STEP 11 ResNet-34 CNN Encoding

Generating the 128-dimensional face fingerprint

This is the culmination of all preprocessing. The fully preprocessed face is passed through a pretrained ResNet-34 Convolutional Neural Network that outputs a 128-dimensional floating-point vector — the face's unique mathematical fingerprint. Faces of the same person produce vectors that are close together in this 128-D space; faces of different people produce vectors far apart.

ResNet-34 Architecture

- 34 weight layers deep
- Key innovation: Residual (skip) connections — each block adds its input directly to its output
- Skip connections solve the vanishing gradient problem that prevented deep networks from training
- Pretrained on 3 million faces from the Labeled Faces in the Wild (LFW) dataset
- Fine-tuned using Triplet Loss for face verification

```
face_encodings = face_recognition.face_encodings(frame_rgb, face_locations)
# Returns: list of numpy arrays, one per detected face
# Each array: shape (128,),  dtype float64
# Values typically in range [-0.5, 0.5]
# np.linalg.norm(encoding) ≈ 1.0  (unit vector)
```

Triplet Loss Training

The model was fine-tuned using Triplet Loss. Each training step uses three images: an Anchor face, a Positive (same person, different photo), and a Negative (different person). The network minimises the distance between Anchor↔Positive while maximising Anchor↔Negative. After training, the 128-D space has a geometric property: same-person embeddings cluster together, different-person embeddings are far apart. This is called metric learning.

STEP 12 Euclidean Distance Matching

Comparing live encoding to stored encodings to verify identity

The final step is computing how similar the live face encoding is to each stored student encoding. We use Euclidean distance — the straight-line distance between two points in 128-dimensional space. The formula sums the squared difference across all 128 dimensions and takes the square root.

```
# d(A, B) = sqrt( Σ (A_i - B_i)²  for i in 0..127 )
distances = face_recognition.face_distance(known_encodings, live_encoding)
# Vectorised NumPy — compares against ALL known faces at once
best_idx  = np.argmin(distances)   # Index of closest match
best_dist = distances[best_idx]
if best_dist <= 0.50:
    name = known_names[best_idx]   # VERIFIED
else:
    name = 'Unknown'               # NOT VERIFIED
```

Distance Range	Meaning	System Result
0.00 – 0.30	Very high confidence — same photo or identical conditions	Verified ✓
0.30 – 0.50	Good match — normal real-world conditions	Verified ✓
0.50 – 0.60	Borderline — could be similar-looking person	Not Verified (safe) ✗
0.60 – 1.00+	Different person	Not Verified ✗

4. Code Explanation — Key Functions

4.1 load_known_faces()

This function runs once at application startup. It reads every image file from the images/ folder, generates a 128-D face encoding for each one, and caches them in memory as global lists for fast comparison during live verification.

```
def load_known_faces():
    for filename in os.listdir(IMAGE_FOLDER):
        img = cv2.imread(filepath)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  # BGR→RGB
        encoding = face_recognition.face_encodings(img)[0]
        known_face_encodings.append(encoding)
        known_names.append(name_from_filename)
```

IndexError Handling

The function wraps face_encodings() in a try-except for IndexError. This handles registration photos where no face was successfully captured (blurry, partially cropped, or too dark). Rather than crashing, it logs a warning and skips that file. This prevents a corrupted images/ folder from breaking the entire system.

4.2 gen_frames() — The Camera Generator

This is the core of the real-time system. It is a Python generator function that continuously yields JPEG-encoded frames from the webcam as a multipart HTTP stream (MJPEG format), which the browser renders as a live video feed. It operates in two modes controlled by the is_registering flag.

Mode	is_registering	Behaviour
Registration Mode	True	Shows prompt overlay on frame. No face processing. Admin sees live feed and clicks capture.
Verification Mode	False	Full 11-step preprocessing pipeline runs. HOG detection → ResNet encoding → Euclidean match → MongoDB log.

4.3 Duplicate Attendance Prevention

Without deduplication, a student standing in front of the camera for 5 seconds at 30 FPS would generate 150 attendance records. The system queries MongoDB for the most recent log for that person+subject combination. If the last entry was within 60 seconds, the record is skipped.

```
last_log  = attendance_collection.find_one(
    {"name": name, "subject": subject},
    sort=[("timestamp", -1)]
)
time_diff = (datetime.now() - last_log['timestamp']).total_seconds()
if time_diff < 60:
```

```
    should_log = False    # Skip — too soon
```

4.4 Flask Routes

Route	Method	Purpose
/	GET	Home page — student selects subject and starts attendance
/verify	GET	Attendance page — displays live MJPEG stream + subject label
/verify_feed	GET	Streams MJPEG video from gen_frames() to the browser
/login	GET/POST	Admin authentication — validates hardcoded credentials
/register_page	GET	Admin registration page for adding new students
/register_feed	GET	MJPEG stream for registration camera preview
/capture_face	POST	Captures frame, saves student image, reloads encodings
/dashboard	GET	Displays attendance logs, filterable by subject

5. Database Design — MongoDB

MongoDB is used as the database because its document-based (NoSQL) structure does not require a fixed schema and is ideal for timestamped attendance logs. Each attendance record is a single document with three fields.

Field	Data Type	Example Value	Description
name	String	MALINDU_KAVISHAN--D/DBA/23/0012	Student name + ID from filename
subject	String	Digital Image Processing	Subject passed from URL parameter
timestamp	DateTime	2025-03-17T10:45:32	Python datetime.now() at verification

Why MongoDB Over SQL?
A relational SQL database would require a fixed schema and JOIN operations to query attendance by subject+date. MongoDB's find() with a subject filter returns results directly in one operation. The flexible document structure also makes it easy to add new fields (e.g., confidence score, camera ID) without altering a table schema.

The dashboard page queries MongoDB with optional subject filtering and displays results sorted by most recent timestamp:

```
records = attendance_collection.find({"subject": subject_filter})
.sort("timestamp", -1)    # Most recent first
```

6. Technical Glossary — Key Terms Explained

Every technical term used in this project is explained below in plain language. Understanding these terms is essential for the interview.

Term	Plain Language Explanation
BGR / RGB	The order of colour channels in an image. BGR means Blue-Green-Red (OpenCV's default). RGB means Red-Green-Blue (standard everywhere else). They contain the same data but in different order — like reading a word forwards vs backwards.
NumPy Array	A grid of numbers stored in computer memory. An image is a 3D NumPy array: (height × width × colour channels). Very fast for mathematical operations because it uses optimised C code under the hood.
HOG (Histogram of Oriented Gradients)	A method to describe the shape of objects in an image. It looks at how much brightness changes and in which direction at every part of the image, then builds a histogram (count chart) of those directions. The result is a numerical fingerprint of the image's shape structure.
SVM (Support Vector Machine)	A trained classifier that draws a dividing line (or hyperplane) between two categories in data. In HOG detection, it was trained to draw a line between 'face' data and 'non-face' data. When the HOG descriptor of a new window falls on the 'face' side, it is flagged as a face.
ResNet-34	A deep neural network with 34 layers. 'Res' stands for Residual — the network uses shortcut connections that skip layers, helping it train deeper without losing gradient signal. Used here to extract a 128-number fingerprint from a face image.
128-D Embedding	A list of 128 decimal numbers that uniquely describes a face. Think of it as GPS coordinates in 128 dimensions. Faces of the same person produce coordinates that are close together; faces of different people produce coordinates far apart.
Euclidean Distance	The straight-line distance between two points. Here, between two 128-number vectors. A small distance means similar faces; a large distance means different faces. We accept distances below 0.50 as a match.
Triplet Loss	A training method where the neural network sees three faces at once: an anchor, the same person (positive), and a different person (negative). It is trained to push the anchor close to positive and push the anchor away from negative. This teaches the network to distinguish faces.
CLAHE	Contrast Limited Adaptive Histogram Equalization. A technique that locally brightens dark areas of an image while preventing already-bright areas from being over-brightened. Used to handle uneven classroom lighting so faces are consistently visible.
LAB Colour Space	An alternative way to represent colours. L = Lightness (brightness), A = green↔red colour, B = blue↔yellow colour. Separating brightness (L) from colour (A,B) lets us adjust brightness without distorting skin tones.
Gaussian Blur	A smoothing filter that replaces each pixel with a weighted average of its neighbours. Weights follow a bell curve (Gaussian). Used to remove camera sensor noise without losing the face's edge structure.
MJPEG Stream	A series of JPEG images sent one after another over HTTP. The browser receives them and displays them as a smooth video. Flask uses a generator function to continuously send frames as multipart HTTP responses.

Term	Plain Language Explanation
Flask Generator	A Python function that uses 'yield' instead of 'return'. It produces values one at a time and keeps running. gen_frames() yields one JPEG frame at a time to the browser, creating the live video effect.
Image Pyramid	A technique where the same detection algorithm is run on the image at multiple sizes (each slightly smaller). This allows detecting faces that are close to the camera (large in the frame) and far away (small in the frame) with the same fixed-size detector window.
Affine Transformation	A mathematical operation that can rotate, scale, and translate an image while keeping straight lines straight. Used in face alignment to rotate a tilted face to a horizontal position based on eye landmark positions.
Metric Learning	Training a neural network not to classify (cat vs dog) but to measure similarity. After metric learning, the network's output space has a geometric property: similar inputs are mapped to nearby coordinates, dissimilar inputs to distant coordinates.
LFW Dataset	Labeled Faces in the Wild — a public dataset of 13,000+ face photographs collected from the internet. Used to train and benchmark face recognition models including the ResNet-34 used in this project.
CAP_DSHOW	A Windows-specific flag for OpenCV. Forces OpenCV to use the DirectShow camera API instead of the default Microsoft Media Foundation (MSMF) API. Fixes black frame and camera initialisation issues on most Windows laptops.
NoSQL / MongoDB	NoSQL means 'Not only SQL' — a database that stores data as flexible documents (like JSON) rather than rigid tables. MongoDB is the most popular NoSQL database. Ideal for this project because each attendance record can have different fields without needing schema changes.

7. Interview Questions & Answers

These are the most likely questions an interviewer will ask based on your project. Study the answers — they cover every technical aspect.

7.1 Project Understanding

Q Can you explain what your project does in simple terms?

A Our system replaces manual attendance with an automated face recognition solution. A webcam continuously scans the room. When a student's face is detected, it is compared against a database of registered student face encodings. If the match confidence is high enough (Euclidean distance below 0.50), their name and the subject are automatically logged into a MongoDB database with a timestamp. The lecturer can view the attendance dashboard from any browser without doing anything manually.

Q Why did you use face_recognition library instead of building your own model?

A Building a face recognition model from scratch requires hundreds of thousands of labelled face images, weeks of GPU training time, and deep ML expertise. The face_recognition library provides a pretrained ResNet-34 model trained on 3 million faces, giving us state-of-the-art accuracy immediately. Our project's focus is on the image preprocessing pipeline and system integration, not on reinventing a solved model. Using a reliable pretrained model lets us focus on the engineering challenges — real-time performance, Windows compatibility, database design, and the web interface.

Q What is the tolerance value 0.50 and how did you choose it?

A The tolerance is the maximum Euclidean distance between two face encodings for them to be considered the same person. Distance 0.0 means identical; 1.0+ means completely different people. We chose 0.50 as a balanced threshold — strict enough to reject similar-looking people (e.g., siblings) while lenient enough to handle normal variation in lighting, glasses, and head angle. Values below 0.45 cause false rejections of valid students; values above 0.55 risk accepting the wrong person. We tested 0.45, 0.50, and 0.55 in our university environment and 0.50 gave the fewest false positives and false negatives.

Q How does the duplicate prevention work?

A Without deduplication, a student standing still for 5 seconds at 30 FPS would generate 150 records. Each time a face is verified, we query MongoDB for the most recent attendance record for that exact name+subject combination using find_one() with sort by timestamp descending. We then calculate the time difference between now and that last record. If it is less than 60 seconds, we skip logging. The 60-second window was chosen because it is long enough to prevent duplicates from a single standing position but short enough to allow re-logging if a student leaves and returns.

7.2 Technical — Preprocessing

Q Why do you convert BGR to RGB? What happens if you skip it?

A OpenCV reads images in BGR (Blue-Green-Red) channel order — a legacy from early video hardware. The face_recognition library and the ResNet-34 model inside it were trained on RGB (Red-Green-Blue) images. If we pass BGR directly, the red and blue channels of every pixel are swapped. A person with warm skin tone (high R in RGB) appears cool-toned (high B in BGR). The CNN generates a completely different 128-D embedding than the one stored during registration, so Euclidean distance will be large and the person will never be verified. The system runs without errors but identifies nobody — a silent failure that is very hard to debug.

Q Why do you resize the frame to 25%? Why not use the full resolution?

A HOG face detection uses a sliding window — a fixed detector window moves across the image at every position. On a 640×480 frame this is thousands of window positions. On 1080p it is tens of thousands. Running this on full resolution at 30 FPS would require a very powerful GPU and would still cause significant lag on a standard laptop. Resizing to 25% reduces pixel count by 94% (640×480 = 307,200 pixels vs 160×120 = 19,200 pixels), making real-time detection feasible on a regular laptop CPU. The accuracy impact is minimal because faces remain detectable at lower resolution — their structural features span many pixels even at 25% size.

Q What is CLAHE and why did you use it instead of simple histogram equalization?

A CLAHE stands for Contrast Limited Adaptive Histogram Equalization. Simple histogram equalization works globally — it redistributes all pixel intensities across the full 0–255 range. This causes problems in mixed-lighting environments: a dark background gets brightened so aggressively that it becomes noisy, while a well-lit face loses contrast. CLAHE solves this by dividing the image into 8×8 tiles and equalising each tile independently. The clipLimit=2.0 prevents any intensity bin from growing too large, preventing noise amplification. We also apply it only to the L (Luminance) channel in LAB colour space — this preserves natural skin tones and eye colours while fixing only the brightness distribution. This is essential in KDU lecture halls where one student may be near a window and another in shadow.

Q What is HOG and how does face detection work?

A HOG (Histogram of Oriented Gradients) describes the shape of objects by analysing image gradients. For every pixel, it computes how much and in which direction the brightness changes. The image is divided into 8×8 cells and a histogram of gradient directions (9 orientation bins) is built for each cell. Groups of 4 cells form a 16×16 block that is L2-normalised for lighting robustness. A trained SVM classifier then slides a 64×128 window across the image — at each position it computes the HOG descriptor and the SVM decides: face or not face. This is repeated at multiple image scales (image pyramid) to detect faces of different sizes. Non-maximum suppression then merges overlapping detections.

Q What is Gaussian Blur and why do you use it?

A Gaussian Blur is a smoothing filter that replaces each pixel's value with a weighted average of its neighbours. The weights follow a 2D Gaussian (bell curve) distribution — closer neighbours have higher weight, farther neighbours have lower weight. We use a 3×3 kernel with $\sigma=0.5$. It removes camera sensor noise: random pixel variations that should be 120 but read as 117 or 124. Without this, those random variations create small spurious intensity gradients that HOG misinterprets as facial features, causing false detections or degraded descriptors. A 3×3 kernel is a mild blur — it removes noise without blurring the actual facial edges that HOG relies on.

7.3 Technical — Deep Learning

Q What is ResNet-34 and why does it have 'residual' connections?

A ResNet-34 is a deep convolutional neural network with 34 weight layers, introduced by He et al. in 2015. The 'residual' part refers to skip connections: each residual block adds the block's input directly to the block's output (output = F(x) + x, where F(x) is what the block learned). This creates shortcut paths for gradients to flow backwards during training. Without skip connections, gradients in very deep networks shrink exponentially as they propagate back through many layers — called the vanishing gradient problem — and the early layers fail to learn. Skip connections bypass this by allowing gradients to flow directly. This is what enables training networks with 34, 50, or even 152 layers effectively.

Q What is the 128-D embedding and what does it represent?

A The 128-dimensional embedding is a vector of 128 decimal numbers that is the ResNet-34 network's output for a given face image. Think of it as GPS coordinates in a 128-dimensional space that represents the mathematical essence of a face. The network was trained so that two photos of the same person, despite different lighting and angles, produce coordinates that are close together (small Euclidean distance). Two photos of different people produce coordinates that are far apart. The network learned this mapping through triplet loss training on 3 million face images. The number 128 was chosen as a balance between expressiveness (enough dimensions to distinguish billions of faces) and efficiency (small enough to compare quickly).

Q What is Triplet Loss and how does it train the model?

A Triplet loss is the training objective used to fine-tune the ResNet-34 for face verification. Each training step uses three images: an Anchor (a reference face), a Positive (same person, different photo), and a Negative (different person). The loss function is: max(0, d(Anchor, Positive) - d(Anchor, Negative) + margin), where d is Euclidean distance and margin (typically 0.2) ensures a minimum gap. The network minimises this loss by learning to push Anchor closer to Positive and push Anchor away from Negative simultaneously. After millions of such training steps, the 128-D embedding space has the geometric property we need for face verification.

Q Why use Euclidean distance for matching? Why not cosine similarity?

A Both work for comparing face embeddings. Euclidean distance measures the straight-line distance between two points in 128-D space. Cosine similarity measures the angle between two vectors. For unit vectors (which ResNet-34 embeddings approximately are — their L2 norm ≈ 1.0), Euclidean distance and cosine similarity produce equivalent ranking results. The face_recognition library uses Euclidean distance and provides empirically validated thresholds (the 0.50 tolerance) specifically for Euclidean distance. Using cosine similarity would require re-calibrating the threshold from scratch.

7.4 System & Architecture

Q Why did you choose MongoDB over a SQL database like MySQL?

A MongoDB's document model is a better fit for attendance logs. Each attendance record is a self-contained document (name, subject, timestamp) that does not need to JOIN with other tables. MongoDB's flexible schema also means we can add new fields — like confidence score or camera ID — to future

records without altering an existing table structure. The PyMongo driver integrates naturally with Python dictionaries and Flask. For a simple time-series log like attendance, MongoDB's find() with a timestamp sort is faster and simpler than equivalent SQL queries with indexes on multiple columns.

Q What is an MJPEG stream and how does it work in your Flask app?

A MJPEG (Motion JPEG) is a video format where each frame is a separate JPEG image. Flask's gen_frames() is a Python generator function that continuously captures frames, encodes them as JPEG, and yields them. Flask wraps this in a multipart/x-mixed-replace HTTP response — a special content type where the server keeps sending new data parts without closing the connection. The browser receives each JPEG part and replaces the previous image, creating the appearance of smooth video. This is simpler than WebRTC or WebSockets and works in all browsers without plugins.

Q What were the Windows compatibility challenges and how did you solve them?

A Four main challenges: (1) Camera black frames — OpenCV's default MSMF backend is incompatible with many Windows webcam drivers. Fixed with cv2.CAP_DSHOW flag. (2) Double camera open — Flask's debug reloader starts the app twice, trying to open the camera twice. Fixed with use_reloader=False. (3) dlib installation — requires C++ compilation, which needs CMake and Visual Studio Build Tools pre-installed on Windows. (4) MongoDB start — on macOS it starts with a Homebrew command; on Windows it requires net start MongoDB in an Administrator CMD or running mongod --dbpath C:\data\db manually.

Q What are the planned improvements and why are they important?

A Five improvements: (1) Multi-frame confirmation — instead of verifying on a single frame, require 3 consecutive frames to agree before logging. Reduces false positives from motion blur. (2) CNN detection model — replace HOG with MTCNN or SSD for better accuracy with angled faces and dim lighting. (3) Excel/CSV export — lecturers need downloadable attendance reports for their records. (4) Anti-spoofing — currently the system could be fooled by holding a printed photo in front of the camera. A liveness detection module (detecting eye blink or head movement) would prevent this. (5) Multi-subject session management — allowing multiple subjects to run simultaneously with different camera streams.

7.5 Challenging Questions

Q What is the accuracy of your system? How would you evaluate it?

A We have not formally evaluated accuracy yet — it is listed as a planned improvement. To evaluate properly, we would collect a test dataset of face images not used in training (different lighting, angles, glasses). We would measure: (1) True Positive Rate (TPR) — percentage of registered students correctly verified; (2) False Positive Rate (FPR) — percentage of unregistered people incorrectly accepted; (3) False Negative Rate (FNR) — percentage of registered students incorrectly rejected. We would plot a ROC curve by varying the 0.50 tolerance threshold to find the optimal operating point. The face_recognition library achieves 99.38% accuracy on the LFW benchmark, but real-world university conditions are harder.

Q Is your system secure? Can it be fooled?

A In its current state, the system has two known vulnerabilities: (1) Photo attack — someone could hold a printed or digital photo of a registered student in front of the camera and get their attendance marked. This

is solved by anti-spoofing — detecting that the face is 2D (no depth) rather than a real 3D person. (2) Admin credential exposure — the admin email and password are hardcoded in the Python file. If the repository were public, these would be exposed. This should be moved to an environment variable (.env file) loaded at runtime. The repository is currently private to mitigate this.

Q Why does converting from BGR to RGB matter for the neural network but not for the HOG detector?

A HOG detection works on image intensity gradients — the magnitude and direction of brightness changes. Brightness is computed from a grayscale conversion of the image, which is a weighted sum of R, G, and B channels. Since the grayscale formula (0.299R + 0.587G + 0.114B) combines all three channels, swapping R and B produces slightly different grayscale values, but the structural gradient patterns of a face remain largely intact. HOG is looking for the shape of a face (eye sockets, nose bridge, jaw) which is present in both BGR and RGB grayscale conversions. ResNet-34, however, operates directly on colour pixel values — it learned specific colour patterns (skin tones, eye colours) in RGB space. Feeding BGR changes every colour feature the network learned, making its output meaningless.

8. Troubleshooting Guide

Problem	Likely Cause	Fix
Face not detected in good lighting	BGR/RGB mixup or CLAHE not applied	Verify cvtColor step; add CLAHE
Known person marked Unknown	Tolerance too strict or poor registration image	Raise tolerance to 0.52; re-register with better photo
Wrong person verified	Tolerance too lenient; similar appearance	Lower tolerance to 0.45; register multiple angles
Black frames on Windows	MSMF camera backend issue	Use cv2.VideoCapture(0, cv2.CAP_DSHOW)
Camera opens twice on startup	Flask debug reloader launching app twice	Set use_reloader=False in app.run()
Very slow processing (< 5 FPS)	Resize not applied or CNN model used	Confirm fx=0.25; switch to model='hog'
MongoDB connection refused	MongoDB service not running	Run: net start MongoDB (as Administrator)
dlib install fails	Missing C++ build tools	Install CMake + Visual Studio Build Tools first

Group 04 | CS31092 — Digital Image Processing
General Sir John Kotelawala Defence University | Faculty of Computing
D/BCE/24/0007 Dissanayake | D/BCS/24/0010 Gajanayaka | D/DBA/23/0012 Kavishan